

Finance AI Assistant

Project Documentation — From Dataset to Deployment

*A retrieval-augmented, tool-using finance agent over real SEC filings, live market data, and news —
built end to end, one deliberate decision at a time.*

Sri Sakticharan

September 2026

1. What This Project Is

The Finance AI Assistant is an AI agent that answers questions about three public companies — Apple, Microsoft, and Nvidia — by combining three real, live sources of information instead of relying on a language model's memory: the companies' actual SEC 10-K filings (via retrieval-augmented generation), live stock prices, and current news. A user asks a plain-language question in a chat interface; the system decides which tools it needs, calls them, and synthesizes a single grounded answer with citations back to the source material.

The project was built from first principles, end to end, specifically as a learning exercise: every layer of the stack — data ingestion, embeddings, vector search, agentic tool-calling, production hardening, containerization, automated testing, CI/CD, and live deployment — was built, broken, debugged, and understood rather than scaffolded from a template. That is the throughline of this document: not just what the system does, but what it took to get it there, including the real bugs hit along the way and the reasoning behind every consequential decision.

A companion file, the Decision Log, captures every decision in a stricter decision → alternatives → why → limitations → interview-answer format, meant as direct interview preparation material. This document is the narrative version — the story of the build, phase by phase, for anyone (including a future version of the author) who wants to understand how the whole system fits together without reading it decision by decision.

System at a Glance

Layer	What it does	Technology
Data source	Real SEC 10-K filings for AAPL, MSFT, NVDA	SEC EDGAR public API
Retrieval	Chunk, embed, and search filing text	Voyage AI embeddings + Qdrant vector DB
Reasoning / agent	Decides which tools to call, synthesizes answers	Gemini (gemini-3.1-flash-lite)
Tools	RAG search, live stock price, ratio/growth calculator, news search	yfinance, Tavily, custom Python
Backend API	Exposes the agent over HTTP	FastAPI (Python)
Frontend	Chat interface	React + TanStack Start (TypeScript)
Packaging	Reproducible runtime	Docker + Docker Compose
Testing	Automated correctness checks	pytest
CI/CD	Test, build, and deploy on every push	GitHub Actions
Hosting	Live, public deployment	Render (API) + Vercel (UI) + Qdrant Cloud (vectors)

2. How a Question Actually Flows Through the System

When a user submits a question in the chat UI, the browser sends it as a JSON POST request to the FastAPI backend's /ask endpoint. FastAPI validates the request shape automatically (via Pydantic models) and hands the raw question straight to `run_agent()`, the single entry point into the agent loop — the HTTP layer does no reasoning of its own.

Inside the agent loop, Gemini receives the question along with a system prompt describing four available tools: `search_filings` (RAG search over the 10-Ks), `get_stock_price` (live market data), the calculator (ratio and percentage-change math), and `search_news` (current news via Tavily). Gemini decides which tool or tools the question actually requires, the orchestrator executes those tool calls, and the raw results are fed back to Gemini for a final synthesis pass — one coherent, cited answer, not a stitched-together list of per-tool outputs.

The RAG path specifically works like this: the user's question is rewritten into filing-style language, embedded with Voyage AI, and used to search Qdrant for the most relevant chunks of 10-K text (filtered by company where applicable). Voyage's reranker re-scores the wider candidate pool against the actual question before the top few chunks are handed to Gemini to write the final answer — retrieval and generation are deliberately separate steps, not one blended call.

Every external call — to Gemini, Voyage, Qdrant, Tavily, or yfinance — passes through centralized, logged, retry-wrapped infrastructure rather than being called ad hoc from wherever it's needed. That design choice, made during the production-hardening phase, is what turns a script that works on a good day into a system that survives a bad one.

3. Phase 1 — The RAG Foundation

The project starts with real regulatory data, not a toy dataset. 10-K filings for Apple, Microsoft, and Nvidia are pulled directly from SEC EDGAR's public API — free, authoritative, and a better learning target than a paid wrapper, since working with EDGAR directly means actually handling CIK lookups, accession numbers, and SEC's rate limiting rather than having a library hide all of that.

The first real obstacle showed up immediately: the initial text extraction produced garbled output — namespace URLs and hidden tag names mixed into the readable filing text. The cause was Inline XBRL (iXBRL), a format where machine-readable financial data is embedded directly inside the same HTML as the human-readable report, hidden from view with CSS. A naive text extraction pulls every text node, visible or not. The fix was to explicitly strip the hidden XBRL elements before extracting text — a small fix, but a genuine "I looked at the raw output and it didn't match what I expected" debugging story, not a hypothetical one.

Filing text is then split into ~400-token chunks using recursive, structure-aware chunking — trying paragraph breaks first, falling back to sentence breaks, and only hard-cutting text as a last resort — with

a 50-token overlap between chunks so meaning doesn't get severed at a boundary. Each chunk is embedded with Voyage AI's voyage-4 model, chosen deliberately over the more commonly recommended voyage-3-large after checking Voyage's own pricing documentation directly: voyage-4 is covered by the free 200-million-token allowance, while voyage-3-large, despite being cited more often in tutorials, is not. The embeddings are stored in Qdrant, self-hosted via Docker at this stage rather than a managed service like Pinecone — a deliberate choice, since operating real infrastructure instead of calling a managed API was one of this project's actual learning goals.

The hardest problem in this phase wasn't a crash — it was a retrieval system that returned a confidently wrong answer instead of a missing one. Asking what employee-related risks Nvidia discloses kept surfacing an unrelated executive biography instead of Nvidia's real Human Capital Management disclosure, even though the real content was confirmed to exist in the filing. Diagnosing this took five real layers of investigation: ruling out cross-company noise, adding LLM-based query rewriting to bridge casual phrasing and filing-style language, widening the candidate pool from the top 5 to the top 15 results, adding a reranking step (Voyage's rerank-2.5) to re-score candidates against the actual question, and finally implementing section-aware chunking so a chunk could no longer span two different 10-K "Item" sections. Each layer measurably improved retrieval quality. One edge case remained even after all of that — "Human Capital Management" turned out to be a subsection inside Item 1, not a separately numbered section, so it still shared a chunk with unrelated content no header-detection logic could see. After investigating a raw-HTML rebuild and finding no reliable signal there either, that edge case was deliberately documented as a known limitation rather than chased indefinitely — a real engineering judgment call about where to stop, not an unresolved bug hidden from view.

4. Phase 2 — Turning RAG Into an Agent

A working retrieval system only answers questions about what's inside the filings. Phase 2 wraps that RAG pipeline as one tool among several, giving the system the ability to decide, per question, what kind of information it actually needs — a philosophy shift from "run this fixed pipeline" to "reason about which capability applies."

Before building the agent loop itself, two real generation-quality bugs surfaced in answer synthesis: answers were getting cut off mid-sentence, and multi-chunk answers read like separate summaries stapled together rather than one synthesized response. Both were fixed with a higher output token ceiling and an explicit prompt instruction to weave sources together rather than summarize each one separately — confirmed by re-running the same test question and comparing the before-and-after output directly, not just assuming the fix worked.

Four tools were built: the existing RAG search, a live stock-price lookup via yfinance (with field names verified against real output rather than trusted from memory), a small calculator exposing two generic functions — ratio and percentage-change — that cover every financial metric needed rather than dozens of named single-purpose functions, and a news search tool via Tavily. Every tool was deliberately kept as a raw data fetcher with no summarization of its own; synthesis happens exactly once, inside the agent's final answer step, so the agent can cross-reference across tools — connecting a stock move to a

specific news event, for example — instead of losing that connection to premature, isolated per-tool summaries.

The most consequential decision in this phase wasn't technical, it was economic: Claude Sonnet 5 was working well, but running it under active development realistically cost \$20 to \$40 a month — incompatible with a hard \$0/month constraint until the project produces income. OpenAI was ruled out for the same reason (no free tier either); Gemini was chosen specifically because it was the only option that actually satisfied the constraint, not because it was judged the strongest model. The migration itself hit a real failure on the first attempt: the newest free-eligible Gemini model, gemini-3.7-flash, allows only 5 requests per minute on its free tier, and a single multi-tool agent turn can burn 2–3 requests by itself — the very first real test failed with a 429 rate-limit error. The fix was choosing a different model with a much higher free quota (gemini-3.1-flash-lite, 30 requests/minute), not adding retry logic to paper over the wrong model choice — and the swap was confirmed not to regress answer quality by re-running the same tests and comparing outputs. The LLM calls were structured so that switching providers again later touches only two or three files, keeping this an explicit, reversible tradeoff rather than a permanent architectural commitment.

5. Phase 3 — Production Hardening

A system that only works when every external API is healthy and every developer is watching the terminal live isn't production-ready — it just hasn't failed yet. Phase 3 closes that gap across three fronts: visibility, resilience, and cost awareness.

Scattered `print()` statements were replaced with a single shared logger writing to both the console and a persistent log file, wired specifically into the two most decision-heavy parts of the system: retrieval scoring (which ticker was detected, how the query was rewritten, what the rerank confidence score was) and the agent's tool-calling loop (which tool was called, with what arguments, what came back, and how many turns the agent took). This turns "why did it answer that way" from a guess into something you can actually read back after the fact.

The rate-limit crash from Phase 2 exposed a deeper, unaddressed risk: any external API call can fail transiently — a rate limit, a timeout, a server hiccup — and the system should survive that rather than crash outright. Rather than bolting retry logic onto three separate files, every Gemini call was centralized behind one wrapper function with automatic exponential-backoff retry (via `tenacity`), so the fix applies everywhere at once and can't be forgotten in a future file. Running entirely on free API tiers with real daily limits also meant needing actual visibility into how close the system was running to those limits — not just a hope that it wouldn't hit them — so a lightweight per-day, per-provider usage tracker was added, verified by running a real multi-tool question and manually confirming the logged counts matched the exact number of underlying API calls made.

6. Phase 4 — Packaging, Testing, and Shipping It

6.1 — API and containerization

The agent was wrapped in a thin FastAPI layer — a GET /health endpoint and a POST /ask endpoint whose entire body is a single call to the already-tested `run_agent()` function. FastAPI's job here is strictly translation, JSON in and out; no new logic lives at the HTTP layer, keeping the same separation of concerns used throughout the project. Pydantic request/response models give automatic input validation for free, and FastAPI's auto-generated interactive docs made the endpoint testable in a browser immediately.

Containerizing the app with Docker and Docker Compose (the app and a Qdrant container running together on one network) immediately caught two real bugs that local development had been silently hiding. First, a hardcoded localhost reference to Qdrant would have broken the moment the app ran inside its own container, since each container's "localhost" refers to itself — fixed proactively with a configurable `QDRANT_HOST` environment variable pointing at Compose's internal service hostname. Second, the very first clean container build crashed on a missing `google-genai` import, even though the app worked perfectly locally — because that dependency, and `tenacity` alongside it, had only ever been installed locally as side effects of earlier commands and were never actually added to `requirements.txt`. A clean Docker build is exactly the kind of forcing function that surfaces "works on my machine" bugs instead of causing them.

6.2 — Automated tests

The test suite was scoped deliberately, not exhaustively: seven pytest tests cover only the pure, dependency-free functions in the codebase — the calculator's ratio and percentage-change logic, and ticker detection. Testing the RAG pipeline or agent loop directly would mean either burning real, rate-limited API calls on every CI run or building a mocking layer — a legitimate next step, deliberately deferred rather than skipped by accident. Setting this up surfaced one more real environment bug: the bare pytest command was silently resolving to a global Python install instead of the project's virtual environment, masking installed packages that were actually present — fixed by invoking `python -m pytest` instead, which pins execution to the interpreter already active in the shell.

6.3 — CI/CD

A GitHub Actions workflow runs on every push to main: checkout, install dependencies, run the full test suite, and build the Docker image, all on a clean `ubuntu-latest` runner with none of the local machine's accumulated state. This was initially scoped honestly as CI, not CD — there was no live deployment target yet, so claiming continuous deployment would have described infrastructure that didn't exist. Once a real deployment target existed (Render), a genuine deploy step was added — with one deliberate catch: Render's default behavior auto-deploys on every push regardless of whether tests pass, which isn't real continuous deployment, just continuous shipping. Auto-deploy was turned off and replaced with a private deploy hook, triggered only as the final step of the workflow and only after tests and the Docker build succeed — confirmed working end to end by checking Render's own event log showed "triggered via deploy hook," not an independent race against CI.

6.4 — Live deployment

The backend runs on Render's free web-service tier, chosen over Fly.io after direct research turned up genuinely conflicting information about whether Fly.io's free tier still exists without a credit card — against a hard \$0/no-subscriptions constraint, the unambiguous option won. Render's free tier spins down after 15 minutes of inactivity, adding a real cold-start delay to the first request after idle time — an accepted tradeoff for a portfolio project, not something a paying-user product could accept as-is.

The vector database was consolidated onto Qdrant Cloud's free tier as the single source of truth for both local development and the live deployment, retiring the local self-hosted Docker instance from Phase 1 once its learning purpose was served — running two copies of the same data was judged a silent-drift risk (re-embed once, forget the second copy, and they quietly disagree) with no real upside once self-hosting had already been learned. Moving to a managed cloud instance surfaced a real behavioral difference: Qdrant Cloud requires an explicit payload index before it will filter by a field like ticker, something self-hosted Qdrant allowed silently — fixed by adding the index explicitly to the ingestion script. Render's free tier has no persistent disk, so Qdrant deliberately does not live there; losing embedded data on every idle restart was not an acceptable tradeoff.

The frontend is a React and TanStack Start chat interface, deployed separately to Vercel, communicating with the FastAPI backend over a CORS-scoped HTTP connection. It presents a clean, terminal-styled console: a sidebar with suggested questions, a live chat thread rendering the agent's markdown-formatted answers, and an About panel describing the project and its backend endpoint.

One last real bug surfaced only after deployment, and only by actually reading an answer critically instead of checking that the pipeline ran: asking for Apple's stock price returned the correct price from the tool, followed by a confident, entirely fabricated paragraph about a CEO transition and regulatory settlement — none of it from any tool. The RAG generation prompt had always explicitly forbidden outside knowledge, but the agent's own system prompt never inherited that same rule, leaving the model free to blend real tool output with its training data at identical confidence. The fix mirrored the RAG prompt's discipline into the agent's system prompt, and the re-test was deliberately built to exercise the exact failure case that caused the bug, not just re-confirm unrelated passing behavior. It's a real, general lesson about agentic systems: a model with both tool access and broad training knowledge will silently blend the two unless explicitly told not to, and the blended answer looks exactly as confident either way.

7. What Was Deliberately Not Built: Kubernetes

Kubernetes was the original final phase of this project and was deliberately dropped. The reasoning was revisited more than once and held up each time. Kubernetes solves problems this project doesn't have: multi-container orchestration, auto-scaling under real load, and self-healing across a cluster of machines. A single Docker container on a real host already handles this project's actual traffic — a personal portfolio demo — correctly and completely.

Cost was checked and ruled out as the real reason: local Kubernetes via Minikube is free, so the usual "Kubernetes is too expensive" argument doesn't actually apply here (that price tag only attaches to

managed cloud Kubernetes like EKS, GKE, or AKS, which run \$500+/month even for a small cluster). The real blocker is architectural — the live deployment already runs on Render, a real, internet-reachable host connected to the actual CI/CD pipeline. A local Minikube cluster would be a second, disconnected environment, unreachable from the internet and outside the real deployment pipeline, running the same container a second time for no benefit to the live system — and would require explicitly telling anyone reviewing the project "these are two unrelated deployments," a more confusing story than simply not having it.

CI/CD, notably, was never treated as similarly optional — automated testing and deployment on every push was treated as close to a baseline expectation for real engineering work regardless of scale, unlike Kubernetes, which is genuinely traffic-dependent. The project was scoped to match its actual requirements rather than to maximize the list of tools touched, and that distinction — knowing what Kubernetes is for and being able to justify not using it, rather than not knowing what it is — is itself part of what this project demonstrates.

8. Cost and Infrastructure Summary

Component	Provider	Tier	Monthly cost
LLM (reasoning/generation)	Google Gemini	gemini-3.1-flash-lite, free tier	\$0
Embeddings + reranking	Voyage AI	voyage-4 + rerank-2.5, free 200M tokens	\$0
Vector database	Qdrant Cloud	Free tier (1GB RAM / 4GB disk)	\$0
News search	Tavily	Free tier	\$0
Market data	yfinance	Free, unofficial Yahoo Finance API	\$0
Backend hosting	Render	Free web service tier	\$0
Frontend hosting	Vercel	Free tier	\$0
CI/CD	GitHub Actions	Free for public/small repos	\$0

Total realistic run cost as of the current build: \$0/month. The one deliberate quality tradeoff behind this number is the LLM choice — Claude or OpenAI would likely produce marginally stronger reasoning, at a real cost of roughly \$20–40/month under active use. The code is structured so reverting that single decision, once there's budget for it, is a small, localized change rather than a rearchitecture.

9. Current State

The system is live end to end: a public chat interface on Vercel talks to a FastAPI backend on Render, which runs the full agent loop against Qdrant Cloud, Gemini, Voyage, Tavily, and yfinance. Every push to

main runs the automated test suite, builds the Docker image, and — only if both succeed — deploys the new backend automatically via a private deploy hook. The project covers three companies (Apple, Microsoft, Nvidia) end to end, including real 10-K citations, live stock prices, current news, and financial-ratio calculations, all through one conversational interface.

Known, accepted limitations are documented rather than hidden: the sub-section chunking gap from Phase 1, Render's cold-start delay after idle time, Qdrant Cloud's free-cluster suspension after a week of inactivity (not yet mitigated with a keep-alive job), and a test suite that deliberately covers only pure functions rather than the full pipeline. Each of these has a clear, specific reason it wasn't fixed further right now, not an unexamined gap.

The companion Decision Log holds the full decision-by-decision detail — every alternative considered, every real bug's root cause and fix, and a ready interview answer for each — for anyone who wants the deeper version of any section above.